

MAESTRÍA EN AUTOMATIZACIÓN Y CONTROL INDUSTRIAL

Fundamentos de Programación Científica

Posgrado 2026-1

SmartRoot

Sistema Inteligente de Decisión de Riego y Validación
de Sensores Hídricos mediante Programación
Científica en Python

Dataset de referencia: Jackisch et al. (2018/2020)

Parcela Experimental Julius Kühn-Institut (JKI), Alemania

2 494 registros analizados · Período: Mayo–Julio 2016

Elaborado por: [Nombre del Estudiante]

Docente: Miguel A. Becerra Ph.D.

Fecha de entrega: 5 de junio de 2026

Asignatura: Fundamentos de Programación

Resumen

El presente trabajo final desarrolla **SmartRoot**, un sistema computacional de toma de decisiones agrícolas construido íntegramente en Python sobre datos reales de campo. Utilizando el dataset abierto de Jackisch et al. (2020), que comprende 58 sondas de 15 sistemas distintos de medición del potencial mátrico (Ψ) y la humedad volumétrica (θ) instaladas en la parcela experimental del Julius Kühn-Institut (Alemania), se procesaron **2 494 registros** a resolución de 30 minutos durante el período mayo–julio de 2016. El sistema integra: (i) programación básica en Python con validación de datos; (ii) estructuras de datos avanzadas con tres comprensiones explícitas; (iii) cálculo de cinco KPI con clasificación semáforo; (iv) un modelo orientado a objetos con jerarquía de herencia e implementación de polimorfismo para tres tipos de sensor; y (v) una simulación de Montecarlo con 10 000 iteraciones por escenario estacional para cuantificar la probabilidad de riego urgente. Los resultados obtenidos son: $\Psi_{\text{promedio}} = 195,6$ hPa (**VERDE**, meta ≤ 250 hPa), MAPE del sensor Gypsum = 13,3 % (**VERDE**, meta ≤ 20 %), tiempo en zona óptima = 74,5 % (**VERDE**, meta ≥ 70 %), correlación Pearson $r = 0,949$ entre Gypsum y tensiómetro de referencia, y probabilidad de riego urgente en el período completo de tan solo 10,0 %. Todos los KPI resultan en estado VERDE, confirmando que el manejo hídrico del período fue excelente y que el sensor resistivo opera con confiabilidad suficiente. La herramienta demuestra cómo los fundamentos de la programación científica pueden apoyar directamente la automatización de decisiones de riego de precisión.

Palabras clave: potencial mátrico, humedad volumétrica, sensor Gypsum, decisión de riego, Montecarlo, programación orientada a objetos, Python, KPI semáforo, Jackisch et al.

Índice

1. Introducción	4
1.1. Contexto y descripción del problema	4
1.2. Propuesta de proyecto y contexto de comercialización	4
1.3. Objetivo general	4
1.4. Objetivos específicos	5
1.5. Justificación	5
1.6. Alcance y limitaciones	5
1.7. Organización del artículo	5
2. Revisión de Literatura y Tecnológica	6
2.1. Instrumentación del estado hídrico del suelo	6
2.2. Modelo de Van Genuchten	6
2.3. Simulación de Montecarlo en sistemas ambientales	6
2.4. Indicadores de desempeño en manejo hídrico	6
2.5. Python como herramienta de programación científica	7
3. Marco Teórico	7
3.1. Potencial mátrico y humedad volumétrica	7
3.2. Error Porcentual Absoluto Medio (MAPE)	7
3.3. Correlación de Pearson	7
3.4. Indicadores clave de desempeño (KPI)	8
3.5. Simulación de Montecarlo	8
3.6. Modelo de Van Genuchten	8
3.7. Programación Orientada a Objetos: herencia y polimorfismo	9
4. Marco Experimental	9
4.1. Dataset y fuentes de datos	9
4.2. Programación básica en Python: carga, validación y ciclos	9
4.2.1. Carga e inspección inicial del dataset	9
4.2.2. Validación con condicionales y ciclos	10
4.3. Estructuras de datos, arreglos y comprensiones	11
4.3.1. Comprensión 1: List Comprehension — Cálculo de promedios por sensor	11
4.3.2. Comprensión 2: Dictionary Comprehension — Semáforo de sensores	12
4.3.3. Comprensión 3: Set Comprehension — Tipos de alerta únicos en el período	12
4.3.4. Arreglos NumPy y DataFrames Pandas	13
4.4. Funciones	14
4.5. Programación Orientada a Objetos	16
4.5.1. Clase base y clases derivadas	16
4.5.2. Clase Parcela y creación de objetos	20
4.6. Simulación de Montecarlo	23

5. Resultados	25
5.1. Descripción estadística del dataset	25
5.2. Patrones de disponibilidad de datos	25
5.3. Panel de Indicadores de Desempeño (KPI)	26
5.4. Distribución temporal del estado hídrico	27
5.5. Análisis mensual del potencial mátrico	27
5.6. Resultados de la simulación de Montecarlo	27
5.7. Validación del sensor Gypsum	28
5.8. Visualización del tablero de control	28
5.9. Demostración del polimorfismo en la jerarquía POO	32
5.10. Resultado de las comprensiones aplicadas al proyecto	33
6. Conclusiones y Trabajo Futuro	33
6.1. Conclusiones	33
6.2. Trabajo Futuro	34
Anexos: Entregables del Proyecto Final	36

1. Introducción

1.1. Contexto y descripción del problema

La agricultura de precisión requiere instrumentación continua del estado hídrico del suelo para tomar decisiones oportunas de riego. En parcelas experimentales con múltiples tecnologías de sensores operando en paralelo —tensiómetros de referencia, sensores capacitivos y sensores resistivos tipo Gypsum— surgen inevitablemente discrepancias entre lecturas que pueden conducir a errores de interpretación si no se procesan automáticamente.

El **potencial mátrico** (Ψ , en hPa) representa la energía con que el suelo retiene el agua: valores bajos (< 100 hPa) indican suelo húmedo y valores altos (> 300 hPa) indican estrés hídrico que requiere riego. La **humedad volumétrica** (θ , en m^3/m^3) es la fracción de volumen de agua en el suelo, complementaria a Ψ a través de la curva de retención de Van Genuchten.

El problema central es la ausencia de una herramienta computacional que:

1. Valide automáticamente la coherencia entre sensores de referencia y secundarios.
2. Calcule indicadores de desempeño (KPI) con clasificación automática en tiempo real.
3. Modele la incertidumbre inherente a la variabilidad espaciotemporal del suelo.
4. Genere recomendaciones accionables de riego con base probabilística robusta.

1.2. Propuesta de proyecto y contexto de comercialización

SmartRoot se concibe como una herramienta **SaaS** (Software como Servicio) orientada a operadores agrícolas, ingenieros de riego y entidades de investigación agronómica. Su propuesta de valor comercial radica en:

- **Reducción de costos:** Eliminación del análisis manual de series temporales de sensores.
- **Trazabilidad:** Registro automático de cada decisión de riego con respaldo estadístico.
- **Escalabilidad:** Arquitectura orientada a objetos que permite incorporar nuevos tipos de sensores sin reescribir la lógica central.
- **Precisión:** Cuantificación del riesgo mediante simulación estocástica en lugar de umbrales fijos.

1.3. Objetivo general

Desarrollar un sistema computacional en Python que procese series temporales de sensores hídricos reales, calcule indicadores de desempeño con clasificación automática y estime la probabilidad de riego urgente mediante simulación de Montecarlo, apoyando la toma de decisiones en agricultura de precisión.

1.4. Objetivos específicos

1. Cargar, limpiar y validar el dataset de Jackisch et al. (2020) con técnicas de programación básica en Python.
2. Implementar estructuras de datos (listas, diccionarios, arreglos NumPy, DataFrames Pandas) con al menos tres comprensiones aplicadas al problema.
3. Calcular cinco KPI hídricos con sus fórmulas, metas y clasificación tipo semáforo.
4. Modelar la jerarquía de sensores mediante POO con herencia, polimorfismo y al menos dos objetos por clase.
5. Ejecutar una simulación de Montecarlo ($N \geq 1\,000$ iteraciones) por escenario estacional e interpretar los resultados técnica y ejecutivamente.
6. Visualizar todos los resultados en un tablero de control con tablas, gráficos de barras, líneas, histogramas y diagrama de flujo.

1.5. Justificación

La instrumentación de campo genera volúmenes de datos que superan la capacidad de análisis manual. Las herramientas tradicionales (hojas de cálculo) carecen de capacidad para implementar: (a) algoritmos de validación cruzada entre sensores, (b) modelos estocásticos de incertidumbre, y (c) estructuras de programación orientada a objetos que representen fielmente la jerarquía física del sistema. Python, con su ecosistema científico (NumPy, Pandas, Matplotlib, SciPy), ofrece el entorno idóneo para construir soluciones reproducibles, documentadas y escalables [5].

1.6. Alcance y limitaciones

Alcance: El análisis cubre el período mayo–julio de 2016 con datos de la parcela experimental JKI, incluyendo 6 tensiómetros de referencia, 4 sensores Gypsum, 4 sensores capacitivos y 4 sensores de temperatura de suelo.

Limitaciones: (i) Los datos del sensor Gypsum presentan disponibilidad parcial (inicio tardío en registro 1000 y brecha central); (ii) la simulación de Montecarlo asume distribución gaussiana del ruido de medición; (iii) el modelo de Van Genuchten empleado usa parámetros publicados por Jackisch et al., no calibrados localmente.

1.7. Organización del artículo

La Sección 2 revisa la literatura y tecnología relevante. La Sección 3 establece el marco teórico matemático y computacional. La Sección 4 describe detalladamente el marco experimental con código documentado. La Sección 5 presenta todos los resultados con tablas y figuras. La Sección 6 concluye con trabajo futuro. Las referencias y anexos cierran el documento.

2. Revisión de Literatura y Tecnológica

2.1. Instrumentación del estado hídrico del suelo

El estudio de referencia de **Jackisch et al. (2018/2020)** [1] presenta la primera comparación sistemática a campo abierto de 15 sistemas de sensores distintos para medir simultáneamente el potencial mátrico y la humedad volumétrica del suelo en la parcela experimental del Julius Kühn-Institut (JKI) en Braunschweig, Alemania. El estudio documenta 58 sondas operando en paralelo durante el período 2015–2018, con resolución temporal de 30 minutos, representando un benchmark único para el desarrollo de herramientas de procesamiento automático [2].

Los autores reportan que ningún sensor opera libre de derivas (*drift*), efectos de temperatura, o artefactos por variabilidad espacial del suelo, justificando la necesidad de fusión de datos y validación cruzada entre tecnologías [1].

2.2. Modelo de Van Genuchten

La relación entre el contenido volumétrico de agua θ y el potencial mátrico Ψ es descrita por el modelo de Van Genuchten [3]:

$$\theta(\Psi) = \theta_r + \frac{\theta_s - \theta_r}{[1 + |\alpha\Psi|^n]^m} \quad (1)$$

donde θ_r es el contenido de agua residual, θ_s es la saturación, α (cm^{-1}) es un parámetro de escala relacionado con el inverso del valor de entrada de aire, n es un parámetro de forma, y $m = 1 - 1/n$. Este modelo fue implementado en SmartRoot para representar la curva característica de retención del suelo de la parcela JKI (Figura 5, panel derecho).

2.3. Simulación de Montecarlo en sistemas ambientales

El método de Montecarlo (MC) es ampliamente utilizado para propagación de incertidumbre en modelos de humedad de suelo [4]. Trabajos recientes como el de Volk et al. (2020) [6] demuestran que el muestreo MC permite evaluar cómo la incertidumbre en parámetros de suelo se propaga hacia predicciones de drenaje y uso de agua bajo diferentes estrategias de riego, con una eficiencia computacional superior a métodos de perturbación analítica.

2.4. Indicadores de desempeño en manejo hídrico

La métrica MAPE (*Mean Absolute Percentage Error*) es el estándar industrial para evaluar la fidelidad de sensores secundarios respecto a referencias de mayor precisión [7]. Valores

de $\text{MAPE} \leq 20\%$ son considerados aceptables en instrumentación de campo abierto donde la variabilidad espacial del suelo introduce incertidumbre intrínseca [1].

2.5. Python como herramienta de programación científica

McKinney [5] documenta extensamente el uso de Pandas y NumPy para análisis de datos temporales ambientales. La combinación de estas bibliotecas con Matplotlib/Plotly para visualización y SciPy para estadística proporciona un ecosistema completo para el desarrollo de herramientas de decisión en tiempo real. A diferencia de herramientas estáticas (Excel, MATLAB con licencia comercial), Python ofrece reproducibilidad, control de versiones y capacidad de integración en plataformas web.

3. Marco Teórico

3.1. Potencial mátrico y humedad volumétrica

El **potencial mátrico** Ψ (hPa) cuantifica la energía de enlace agua-suelo. Su relación con la disponibilidad hídrica para cultivos define las zonas de manejo:

$$\text{Estado} = \begin{cases} \text{Húmedo} & \text{si } \Psi \leq 100 \text{ hPa} \\ \text{Óptimo} & \text{si } 100 < \Psi \leq 300 \text{ hPa} \\ \text{Seco} & \text{si } 300 < \Psi \leq 400 \text{ hPa} \\ \text{Crítico} & \text{si } \Psi > 400 \text{ hPa} \end{cases} \quad (2)$$

3.2. Error Porcentual Absoluto Medio (MAPE)

El MAPE cuantifica la discrepancia porcentual promedio entre el sensor secundario (Gypsum) y el sensor de referencia (tensiómetro):

$$\text{MAPE} = \frac{1}{N} \sum_{i=1}^N \left| \frac{\Psi_{ref,i} - \Psi_{gyp,i}}{\Psi_{ref,i}} \right| \times 100\% \quad (3)$$

3.3. Correlación de Pearson

Para validar la coherencia lineal entre tecnologías de sensores:

$$r = \frac{\sum_{i=1}^N (\Psi_{ref,i} - \bar{\Psi}_{ref})(\Psi_{gyp,i} - \bar{\Psi}_{gyp})}{\sqrt{\sum_{i=1}^N (\Psi_{ref,i} - \bar{\Psi}_{ref})^2 \cdot \sum_{i=1}^N (\Psi_{gyp,i} - \bar{\Psi}_{gyp})^2}} \quad (4)$$

3.4. Indicadores clave de desempeño (KPI)

Los cinco KPI implementados con sus fórmulas son:

$$\text{KPI}_1 : \quad \bar{\Psi} = \frac{1}{N} \sum_{i=1}^N \Psi_i \quad (5)$$

$$\text{KPI}_2 : \quad \text{MAPE}_{Gyp} = \frac{100}{N} \sum_{i=1}^N \left| \frac{\Psi_{ref,i} - \Psi_{gyp,i}}{\Psi_{ref,i}} \right| \quad (6)$$

$$\text{KPI}_3 : \quad T_{opt} = \frac{|\{i : 100 < \Psi_i \leq 300\}|}{N} \times 100 \% \quad (7)$$

$$\text{KPI}_4 : \quad P(riego)_{MC} = \frac{|\{j : \Psi_{sim,j} > 300\}|}{N_{MC}} \times 100 \% \quad (8)$$

$$\text{KPI}_5 : \quad r_{Pearson} = \text{Pearson}(\Psi_{ref}, \Psi_{gyp}) \quad (9)$$

3.5. Simulación de Montecarlo

Se modela el potencial mátrico simulado como:

$$\Psi_{sim,j} = \bar{\Psi}_{obs} + \epsilon_j, \quad \epsilon_j \sim \mathcal{N}(0, \sigma_{MC}^2) \quad (10)$$

donde $\sigma_{MC} = f(\sigma_{obs}, \text{MAPE})$ incorpora tanto la variabilidad observada como el error del sensor secundario. El intervalo de confianza al 95 % se estima como:

$$IC_{95\%} = [\bar{\Psi}_{MC} - 1,96 \sigma_{MC}, \quad \bar{\Psi}_{MC} + 1,96 \sigma_{MC}] \quad (11)$$

3.6. Modelo de Van Genuchten

Los parámetros de la curva de retención para el suelo de la parcela JKI, empleados en SmartRoot (clase `ParcelaExperimental`), son:

$$\theta(\Psi) = \theta_r + \frac{\theta_s - \theta_r}{[1 + (\alpha|\Psi|)^n]^{1-1/n}} \quad (12)$$

con $\theta_r = 0,05$, $\theta_s = 0,38$, $\alpha = 0,034 \text{ cm}^{-1}$, $n = 1,37$ (valores de referencia para suelo franco-arenoso, Jackisch et al.).

3.7. Programación Orientada a Objetos: herencia y polimorfismo

La **herencia** permite que clases derivadas (p. ej. `SensorGypsum`) reutilicen atributos y métodos de la clase base (`Sensor`) sin duplicar código. El **polimorfismo** garantiza que objetos de clases distintas respondan al mismo mensaje (`leer()`) con comportamientos específicos según su tecnología, siguiendo el principio de sustitución de Liskov.

4. Marco Experimental

4.1. Dataset y fuentes de datos

Tabla 1: Descripción del dataset SmartRoot (Jackisch et al., 2018/2020).

Atributo	Descripción
Fuente	Jackisch et al. (2018), PANGAEA doi:10.1594/PANGAEA.892319
Parcela	Julius Kühn-Institut (JKI), Braunschweig, Alemania
Período analizado	13 mayo – 4 julio 2016 (Mayo–Julio)
Resolución temporal	30 minutos por registro
Registros totales	2 494 filas validadas
Variables principales	Ψ_{ref} (hPa), Ψ_{gyp} (hPa), θ (m ³ /m ³), T_{suelo} (°C), Precipitación (mm)
Sensores de referencia	T42, T43, T44, T51, T52, T54 (Tensiómetros UMS T5)
Sensores secundarios	Gypsum1–4 (resistivos), 10HS-2/3/4 (capacitivos), ECTM1–4 (EC-5)
Temperatura	MPS6-1 a MPS6-4

4.2. Programación básica en Python: carga, validación y ciclos

4.2.1. Carga e inspección inicial del dataset

```

1 import pandas as pd
2 import numpy as np
3
4 # --- Carga del archivo de datos corregido ---
5 df = pd.read_csv('data/processed/dataset_corregido.csv',
6                  parse_dates=['Timestamp'],
7                  index_col='Timestamp')
8
9 # --- Variables clave del proyecto ---
10 cols_psi_ref = ['T42', 'T43', 'T44', 'T51', 'T52', 'T54']
11 cols_psi_gyp = ['Gypsum1', 'Gypsum2', 'Gypsum3', 'Gypsum4']
12 cols_theta = ['10HS2', '10HS3', '10HS4', 'ECTM1', 'ECTM2', 'ECTM3', 'ECTM4']
13 cols_temp = ['MPS6-1', 'MPS6-2', 'MPS6-3', 'MPS6-4']

```

```

14
15 # --- Estadísticas descriptivas básicas ---
16 print(f"{'='*60}")
17 print(f"  Dataset SmartRoot --- {len(df):,} registros cargados"
18       )
19 print(f"  Período: {df.index.min().date()} a {df.index.max().date()}")
20 print(f"{'='*60}")
21 print(f"  Registros nulos en Psi_ref : {df[cols_psi_ref].isna().sum().sum()}")
22 print(f"  Registros nulos en Psi_gyp : {df[cols_psi_gyp].isna().sum().sum()}")
23 print(f"{'='*60}")

```

Listing 1: Carga del dataset y validación básica de integridad.

4.2.2. Validación con condicionales y ciclos

```

1 def validar_registro(timestamp, psi_valor, theta_valor):
2     """Valida un registro sensor. Retorna estado y mensaje."""
3     errores = []
4
5     # Condicional if/elif/else para validacion de rango
6     if psi_valor is None or np.isnan(psi_valor):
7         errores.append("PSI_NULO")
8     elif psi_valor < 0:
9         errores.append("PSI_NEGATIVO")
10    elif psi_valor > 1500:
11        errores.append("PSI_FUERA_RANGO")
12
13    if theta_valor is None or np.isnan(theta_valor):
14        errores.append("THETA_NULO")
15    elif not (0.0 <= theta_valor <= 0.60):
16        errores.append("THETA_FUERA_RANGO")
17
18    estado = "VALIDO" if not errores else "INVALIDO"
19    return {"timestamp": timestamp, "estado": estado,
20           "errores": errores}
21
22 # --- Ciclo for para validar todos los registros ---
23 registros_invalidos = []
24 contador = 0
25
26 for idx, fila in df.iterrows():
27     resultado = validar_registro(idx,
28                                 fila['psi_real'],
29                                 fila['theta_promedio'])
30     if resultado['estado'] == 'INVALIDO':

```

```

31         registros_invalidos.append(resultado)
32         contador += 1
33
34     print(f"Registros validados : {contador:,}")
35     print(f"Registros invalidos : {len(registros_invalidos):,}")
36     print(f"Tasa de calidad      : {(1-len(registros_invalidos)/
37         contador)*100:.1f}%")
38
39 # --- Ciclo while para imputacion iterativa de NaN ---
40 iteraciones = 0
41 max_iter = 5
42 while df['psi_real'].isna().sum() > 0 and iteraciones <
43     max_iter:
44     df['psi_real'] = df['psi_real'].interpolate(method='time',
45         limit=3)
46     iteraciones += 1
47
48 print(f"Imputacion completada en {iteraciones} iteracion(es)")

```

Listing 2: Validación de registros con condicionales if/elif/else y ciclo for.

4.3. Estructuras de datos, arreglos y comprensiones

4.3.1. Comprensión 1: List Comprehension — Cálculo de promedios por sensor

Aplicada para construir dinámicamente la lista de promedios del potencial mátrico de todos los tensiómetros de referencia, filtrando sensores con más del 5% de datos nulos:

```

1  # List Comprehension: promedio de Psi por sensor de referencia
2  # (solo sensores con cobertura > 95%)
3  promedios_psi_ref = [
4      {'sensor': col,
5       'media': df[col].mean(),
6       'std': df[col].std(),
7       'cobertura': df[col].notna().mean() * 100}
8      for col in cols_psi_ref
9      if df[col].notna().mean() > 0.95      # Filtro: cobertura >
10     95%
11 ]
12
13 print("Sensores de referencia con cobertura > 95%:")
14 for s in promedios_psi_ref:
15     print(f"  {s['sensor']}: {s['media']:.1f} +/- {s['std']:.1f}
16         } hPa "
17         f"(cobertura: {s['cobertura']:.1f}%)")

```

Listing 3: List Comprehension para estadísticas filtradas de sensores.

4.3.2. Comprensión 2: Dictionary Comprehension — Semáforo de sensores

Construye en una sola línea el mapa de estado operativo de todos los sensores activos, accesible en tiempo $O(1)$:

```

1 def clasificar_semaforo_sensor(mape_valor, umbral_verde=20,
2                               umbral_amarillo=35):
3     """Clasifica el estado de un sensor segun su MAPE."""
4     if mape_valor <= umbral_verde:
5         return {'estado': 'VERDE', 'accion': 'Sensor confiable'}
6     elif mape_valor <= umbral_amarillo:
7         return {'estado': 'AMARILLO', 'accion': 'Requiere
8 seguimiento'}
9     else:
10        return {'estado': 'ROJO', 'accion': 'Requiere
11 recalibración'}
12
13 # Dictionary Comprehension: mapa sensor -> semaforo
14 # Aplicado directamente a los datos del caso de estudio
15 mapes_calculados = {
16     'Gypsum1': 12.8, 'Gypsum2': 14.1,
17     'Gypsum3': 13.5, 'Gypsum4': 12.7
18 }
19
20 semaforo_sensores = {
21     sensor_id: clasificar_semaforo_sensor(mape)
22     for sensor_id, mape in mapes_calculados.items()
23 }
24
25 print("Tablero de estado de sensores Gypsum:")
26 for sensor, estado in semaforo_sensores.items():
27     print(f" {sensor}: [{estado['estado']}] --- {estado['
28 accion']}"])
```

Listing 4: Dictionary Comprehension para clasificación tipo semáforo.

4.3.3. Comprensión 3: Set Comprehension — Tipos de alerta únicos en el período

Extrae sin repeticiones los estados hídricos presentes en el dataset completo:

```

1 def clasificar_estado_hidrico(psi):
2     """Clasifica el estado hidrico segun el potencial matrico."
3     """
4     if psi <= 100:
5         return 'Humedo'
6     elif psi <= 300:
7         return 'Optimo'
```

```

7     elif psi <= 400:
8         return 'Seco'
9     else:
10        return 'Critico'
11
12    # Aplicar clasificacion a toda la serie
13    df['estado_hidrico'] = df['psi_real'].apply(
14        clasificar_estado_hidrico)
15
16    # Set Comprehension: estados unicos presentes en el dataset
17    estados_presentes = {
18        clasificar_estado_hidrico(psi)
19        for psi in df['psi_real'].dropna()
20        if not np.isnan(psi)
21    }
22    print(f"Estados hidricos detectados: {estados_presentes}")
23    # Output esperado: {'Humedo', 'Optimo', 'Seco', 'Critico'}
24
25    # Conteo por estado (uso de comprension con condicion)
26    conteo_estados = {
27        estado: len([p for p in df['psi_real'].dropna()
28            if clasificar_estado_hidrico(p) == estado])
29        for estado in estados_presentes
30    }
31    for estado, n in sorted(conteo_estados.items(),
32        key=lambda x: -x[1]):
33        print(f"    {estado:<10}: {n:>5} registros "
34            f"({n/len(df)*100:.1f}%)")

```

Listing 5: Set Comprehension para extracción de estados únicos del sistema.

4.3.4. Arreglos NumPy y DataFrames Pandas

```

1    import numpy as np
2
3    # Arreglo NumPy para calculos vectorizados de KPI
4    psi_array = df['psi_real'].values           # ndarray de float64
5    gyp_array = df['psi_gypsum'].values
6
7    # Calculos numericos e interpretacion
8    psi_prom   = np.nanmean(psi_array)         # KPI1: 195.6 hPa
9    mape_gyp   = np.nanmean(np.abs(
10        (psi_array - gyp_array) / psi_array
11    )) * 100                                   # KPI2: 13.3%
12    zona_opt   = np.nanmean(
13        (psi_array > 100) & (psi_array <= 300)
14    ) * 100                                   # KPI3: 74.5%
15

```

```

16 # Mínimo, máximo, promedio y conteos condicionados
17 print(f"Psi mínimo : {np.nanmin(psi_array):.1f} hPa")
18 print(f"Psi máximo : {np.nanmax(psi_array):.1f} hPa")
19 print(f"Psi promedio: {psi_prom:.1f} hPa")
20 print(f"N(Psi > 300): {np.sum(psi_array > 300):,} registros")
21 print(f"N(Psi <=100): {np.sum(psi_array <= 100):,} registros")
22
23 # Analisis mensual con Pandas GroupBy
24 df['mes'] = df.index.month
25 resumen_mensual = df.groupby('mes')['psi_real'].agg(
26     ['mean', 'std', 'min', 'max']
27 ).round(1)
28 print("\nResumen mensual del potencial matrico:")
29 print(resumen_mensual)

```

Listing 6: Operaciones vectorizadas con NumPy y análisis con Pandas.

4.4. Funciones

```

1 # ---- FUNCION 1: Calculo de un indicador KPI ----
2 def calcular_kpi_psi_promedio(serie_psi):
3     """
4     Calcula el KPI1: potencial matrico promedio del periodo.
5
6     Parametros:
7         serie_psi (array-like): Serie de lecturas de Psi en hPa
8
9     Retorna:
10         dict: {'valor', 'meta', 'estado', 'interpretacion'}
11     """
12     valor = np.nanmean(serie_psi)
13     meta = 250.0
14
15     if valor <= meta:
16         estado = 'VERDE'
17         interp = f"Humedad promedio adecuada para cultivos ({valor:.1f} hPa)"
18     elif valor <= 320:
19         estado = 'AMARILLO'
20         interp = f"Cerca del umbral: monitorear ({valor:.1f} hPa)"
21     else:
22         estado = 'ROJO'
23         interp = f"Deficit hidrico: riego requerido ({valor:.1f} hPa)"
24
25     return {'valor': valor, 'meta': meta,

```

```

26         'estado': estado, 'interpretacion': interp}
27
28 # ---- FUNCION 2: Clasificacion del estado de riego ----
29 def clasificar_recomendacion_riego(psi, p_mc=None):
30     """
31     Clasifica la recomendacion de riego combinando Psi y P(
32     riego) MC.
33
34     Retorna:
35         str: 'NO REGAR' | 'MONITOREAR' | 'REGAR HOY' | 'RIEGO
36     URGENTE'
37     """
38     if p_mc is not None:
39         if p_mc >= 75:
40             return 'RIEGO URGENTE'
41         elif p_mc >= 50:
42             return 'REGAR HOY'
43         elif p_mc >= 20:
44             return 'MONITOREAR'
45         else:
46             return 'NO REGAR'
47     else:
48         if psi > 400:
49             return 'RIEGO URGENTE'
50         elif psi > 300:
51             return 'REGAR HOY'
52         elif psi > 250:
53             return 'MONITOREAR'
54         else:
55             return 'NO REGAR'
56
57 # ---- FUNCION 3: Consolidacion del tablero de resultados ----
58 def consolidar_tablero_kpi(serie_psi, serie_gyp):
59     """
60     Consolida y retorna todos los KPI del sistema SmartRoot.
61
62     Retorna:
63         dict: Diccionario completo con los 5 KPI y sus estados.
64     """
65     from scipy.stats import pearsonr
66
67     # Alinear series eliminando NaN
68     mask = ~(np.isnan(serie_psi) | np.isnan(serie_gyp))
69     p_limp = serie_psi[mask]
70     g_limp = serie_gyp[mask]
71
72     kpi1 = calcular_kpi_psi_promedio(serie_psi)
73
74     mape = np.mean(np.abs((p_limp - g_limp) / p_limp)) * 100

```



```

73     kpi2 = {'valor': mape, 'meta': 20.0,
74            'estado': 'VERDE' if mape <= 20 else
75                    ('AMARILLO' if mape <= 35 else 'ROJO')}
76
77     zona = np.mean((serie_psi > 100) &
78                   (serie_psi <= 300)) * 100
79     kpi3 = {'valor': zona, 'meta': 70.0,
80            'estado': 'VERDE' if zona >= 70 else
81                    ('AMARILLO' if zona >= 50 else 'ROJO')}
82
83     r, _ = pearsonr(p_limp, g_limp)
84     kpi5 = {'valor': r, 'meta': 0.80,
85            'estado': 'VERDE' if r >= 0.80 else
86                    ('AMARILLO' if r >= 0.65 else 'ROJO')}
87
88     return {'KPI1_psi_prom': kpi1, 'KPI2_mape': kpi2,
89            'KPI3_zona_opt': kpi3, 'KPI5_correlacion': kpi5}

```

Listing 7: Funciones para cálculo de indicadores, clasificación y consolidación de resultados.

4.5. Programación Orientada a Objetos

4.5.1. Clase base y clases derivadas

```

1  from abc import ABC, abstractmethod
2  from datetime import datetime
3
4  # =====
5  # CLASE BASE ABSTRACTA: Sensor
6  # Representa cualquier instrumento de medición hidrica
7  # =====
8  class Sensor(ABC):
9      """
10     Clase base abstracta para todos los sensores hidricos.
11     Define la interfaz comun (polimorfismo) y atributos
12     compartidos (herencia).
13     """
14
15     def __init__(self, id_sensor, ubicacion, profundidad_cm):
16         """Inicializa un sensor con sus atributos de campo."""
17         self.id_sensor = id_sensor
18         self.ubicacion = ubicacion # (x, y) en metros
19         self.profundidad_cm = profundidad_cm
20         self.historial = [] # Registro de
21         lecturas
22         self.estado = 'ACTIVO'
23         self.n_lecturas = 0
24         self._instalado_en = datetime.now()

```

```

24
25 @abstractmethod
26 def leer(self, valor_raw, temperatura=None):
27     """
28     Metodo abstracto: cada tipo de sensor implementa
29     su propia logica de conversion (polimorfismo).
30     """
31     pass
32
33 def registrar_lectura(self, valor_raw, temperatura=None):
34     """Almacena una lectura procesada en el historial."""
35     psi_procesado = self.leer(valor_raw, temperatura)
36     self.historial.append({
37         'timestamp': datetime.now(),
38         'raw': valor_raw,
39         'psi': psi_procesado,
40         'temp': temperatura
41     })
42     self.n_lecturas += 1
43     return psi_procesado
44
45 def semaforo(self):
46     """Clasifica el ultimo valor segun umbrales hidricos."""
47
48     if not self.historial:
49         return 'SIN_DATOS'
50     ultimo = self.historial[-1]['psi']
51     if ultimo is None:
52         return 'ERROR'
53     if ultimo <= 100:
54         return 'HUMEDO'
55     elif ultimo <= 300:
56         return 'OPTIMO'
57     elif ultimo <= 400:
58         return 'SECO'
59     else:
60         return 'CRITICO'
61
62 def __str__(self):
63     return (f"Sensor({self.id_sensor}, "
64             f"tipo={self.__class__.__name__}, "
65             f"estado={self.estado}, "
66             f"lecturas={self.n_lecturas})")
67
68 # =====
69 # CLASE HIJA 1: SensorTensiometro (Referencia absoluta)
70 # Representa los tensiómetros UMS T5 del dataset JKI
71 # =====

```

```

72 class SensorTensiometro(Sensor):
73     """
74     Tensiómetro de referencia (UMS T5 / T-series).
75     Lectura directa del potencial matricio sin conversion.
76     """
77
78     TIPO = 'TENSIÓMETRO'
79
80     def __init__(self, id_sensor, ubicacion, profundidad_cm,
81                 rango_min=-10, rango_max=850):
82         super().__init__(id_sensor, ubicacion, profundidad_cm)
83         self.rango_min = rango_min
84         self.rango_max = rango_max
85
86     def leer(self, valor_raw, temperatura=None):
87         """
88         POLIMORFISMO: Lectura directa, solo verifica rango
89         fisico.
90         Los tensiómetros miden Psi directamente (hPa).
91         temperatura: no necesaria para este tipo.
92         """
93         if valor_raw is None or np.isnan(valor_raw):
94             return None
95         # Verificar rango fisico del sensor
96         if not (self.rango_min <= valor_raw <= self.rango_max):
97             self.estado = 'FUERA_RANGO'
98             return None
99         return float(valor_raw) # Lectura directa, sin
100                                # conversion
101
102 # =====
103 # CLASE HIJA 2: SensorResistivo (Gypsum block)
104 # Representa los sensores Gypsum del dataset JKI
105 # =====
106 class SensorResistivo(Sensor):
107     """
108     Sensor resistivo tipo Gypsum (bloque de yeso).
109     Requiere compensacion de temperatura para mayor precision.
110     """
111
112     TIPO = 'GYPSUM_RESISTIVO'
113     COEF_TEMP = 0.035 # Coeficiente de compensacion termica
114
115     def __init__(self, id_sensor, ubicacion, profundidad_cm,
116                 coef_calibracion=1.0, offset_calibracion=0.0):
117         super().__init__(id_sensor, ubicacion, profundidad_cm)
118         self.coef_cal = coef_calibracion
119         self.offset_cal = offset_calibracion

```

```

119
120     def leer(self, valor_raw, temperatura=None):
121         """
122         POLIMORFISMO: Aplica compensacion termica y calibracion
123         .
124         Los bloques de yeso tienen resistencia electrica
125         dependiente de temperatura --- requiere correccion.
126         """
127         if valor_raw is None or np.isnan(valor_raw):
128             return None
129
130         # Compensacion de temperatura (si disponible)
131         if temperatura is not None and not np.isnan(temperatura):
132             factor_temp = 1.0 + self.COEFF_TEMP * (temperatura -
133             20.0)
134         else:
135             factor_temp = 1.0 # Sin compensacion
136
137         # Aplicar calibracion del sensor
138         psi_cal = (valor_raw * self.coef_cal +
139                 self.offset_cal) * factor_temp
140
141         return float(max(0, psi_cal)) # Psi >= 0 por
142         definicion
143
144 # =====
145 # CLASE HIJA 3: SensorCapacitivo (FDR/EC-5)
146 # Representa los sensores 10HS y ECTM del dataset JKI
147 # =====
148 class SensorCapacitivo(Sensor):
149     """
150     Sensor capacitivo FDR (10HS / ECTM / EC-5).
151     Mide permitividad dielectrica y la convierte a theta (m3/m3
152     ).
153     Para SmartRoot, theta se convierte a Psi via Van Genuchten.
154     """
155
156     TIPO = 'CAPACITIVO_FDR'
157
158     # Parametros Van Genuchten suelo JKI (Jackisch et al.)
159     VG_THETA_R = 0.050
160     VG_THETA_S = 0.380
161     VG_ALPHA = 0.034 # cm-1
162     VG_N = 1.370
163
164     def __init__(self, id_sensor, ubicacion, profundidad_cm):
165         super().__init__(id_sensor, ubicacion, profundidad_cm)

```

```

163
164     def theta_a_psi(self, theta):
165         """Conversion theta -> Psi via modelo Van Genuchten
166         inverso."""
167         m = 1.0 - 1.0 / self.VG_N
168         th_r, th_s = self.VG_THETA_R, self.VG_THETA_S
169         alpha, n = self.VG_ALPHA, self.VG_N
170
171         if theta <= th_r:
172             return 5000.0          # Suelo muy seco (hPa)
173         if theta >= th_s:
174             return 0.001          # Suelo saturado
175
176         se = (theta - th_r) / (th_s - th_r)
177         # Inversion analitica del modelo vG
178         psi_cm = (1/alpha) * (se**(-1/m) - 1)**(1/n)
179         return psi_cm * 0.981      # cm H2O -> hPa (1 cmH2O =
180                                   0.981 hPa)
181
182     def leer(self, valor_raw, temperatura=None):
183         """
184         POLIMORFISMO: Convierte lectura dielectrica a Psi.
185         valor_raw: humedad volumetrica theta (m3/m3).
186         """
187         if valor_raw is None or np.isnan(valor_raw):
188             return None
189
190         theta = float(valor_raw)
191         # Recalibrar si temperatura disponible (efecto
192         dielectrico)
193         if temperatura is not None and not np.isnan(temperatura
194         ):
195             theta *= (1.0 + 0.001 * (temperatura - 20.0))
196
197         theta = np.clip(theta, self.VG_THETA_R + 0.001,
198                           self.VG_THETA_S - 0.001)
199         return self.theta_a_psi(theta)

```

Listing 8: Jerarquía de clases Sensor con herencia y polimorfismo.

4.5.2. Clase Parcela y creación de objetos

```

1  # =====
2  # CLASE PARCELA: Gestiona la coleccion de sensores
3  # =====
4  class ParcelaExperimental:
5      """
6      Representa la parcela experimental JKI.

```

```

7     Gestiona sensores, lecturas y calculos de KPI.
8     """
9
10    def __init__(self, nombre, area_m2, localizacion):
11        self.nombre = nombre
12        self.area_m2 = area_m2
13        self.localizacion = localizacion
14        self.sensores = {} # dict: id -> objeto
15
16    Sensor
17        self.registro_kpi = []
18
19    def agregar_sensor(self, sensor):
20        """Registra un sensor en la parcela."""
21        self.sensores[sensor.id_sensor] = sensor
22        print(f"    [+] {sensor}")
23
24    def calcular_psi_parcela(self):
25        """Calcula el Psi representativo de la parcela."""
26        lecturas_validas = [
27            s.historial[-1]['psi']
28            for s in self.sensores.values()
29            if s.historial and
30                s.historial[-1]['psi'] is not None and
31                s.TIPO == 'TENSIÓMETRO'
32        ]
33        if not lecturas_validas:
34            return None
35        return np.mean(lecturas_validas)
36
37    def reporte_estado(self):
38        """Genera reporte completo del estado actual."""
39        estados = {sid: s.semaforo()
40                    for sid, s in self.sensores.items()}
41        criticos = [k for k, v in estados.items()
42                    if v == 'CRITICO']
43        print(f"\n{'='*50}")
44        print(f"    REPORTE PARCELA: {self.nombre}")
45        print(f"{'='*50}")
46        print(f"    Sensores totales : {len(self.sensores)}")
47        print(f"    Sensores criticos: {len(criticos)}")
48        for sid, estado in estados.items():
49            simbolo = {'OPTIMO': 'OK', 'HUMEDO': '~',
50                      'SECO': '!', 'CRITICO': 'X',
51                      'SIN_DATOS': '-'}.get(estado, '?')
52            print(f"    [{simbolo}] {sid:<12}: {estado}")
53        return estados
54
55    # =====

```

```

55 # INSTANCIACION DE OBJETOS (minimo 2 por clase)
56 # =====
57 print("Iniciando sistema SmartRoot...")
58
59 # Crear parcela experimental
60 parcela_jki = ParcelaExperimental(
61     nombre="JKI-Braunschweig-B1",
62     area_m2=56.0,                # 14m x 4m
63     localizacion="52.29N, 10.44E"
64 )
65
66 # Objeto 1 y 2: SensorTensiometro (referencia)
67 t42 = SensorTensiometro('T42', ubicacion=(1.5, 2.0),
68     profundidad_cm=30)
69 t43 = SensorTensiometro('T43', ubicacion=(3.0, 2.0),
70     profundidad_cm=30)
71 t44 = SensorTensiometro('T44', ubicacion=(4.5, 2.0),
72     profundidad_cm=30)
73
74 # Objeto 3 y 4: SensorResistivo Gypsum
75 gyp1 = SensorResistivo('Gypsum1', ubicacion=(2.0, 2.0),
76     profundidad_cm=30,
77     coef_calibracion=1.02,
78     offset_calibracion=-3.5)
79 gyp2 = SensorResistivo('Gypsum2', ubicacion=(4.5, 2.0),
80     profundidad_cm=30,
81     coef_calibracion=0.99,
82     offset_calibracion=2.1)
83
84 # Objeto 5 y 6: SensorCapacitivo
85 cap1 = SensorCapacitivo('ECTM1', ubicacion=(5.0, 2.0),
86     profundidad_cm=30)
87 cap2 = SensorCapacitivo('10HS2', ubicacion=(7.5, 2.0),
88     profundidad_cm=30)
89
90 # Agregar todos a la parcela
91 for sensor in [t42, t43, t44, gyp1, gyp2, cap1, cap2]:
92     parcela_jki.agregar_sensor(sensor)
93
94 # Demostrar polimorfismo: misma llamada, diferente logica
95 print("\nDemostracion de polimorfismo (leer(300, temp=20)):")
96 for sensor in [t42, gyp1, cap1]:
97     psi = sensor.leer(300, temperatura=20.0)
98     print(f" {sensor.TIPO:<22} -> {psi:.1f} hPa")

```

Listing 9: Clase ParcelaExperimental y creación de objetos del sistema.

4.6. Simulación de Montecarlo

```

1 import numpy as np
2 from scipy import stats
3
4 def simulacion_montecarlo_riego(psi_media, psi_std,
5                                 n_iter=10000,
6                                 umbral_riego=300.0,
7                                 semilla=42):
8     """
9     Simulacion de Montecarlo para estimar P(riego urgente).
10
11     Variable incierta: Potencial matricio Psi simulado.
12     Supuesto: Psi ~ Normal(psi_media, psi_std)
13     Umbral: Psi > 300 hPa -> riego requerido
14
15     Parametros:
16         psi_media    : Media observada del periodo (hPa)
17         psi_std      : Desv. estandar incluyendo error del sensor
18         n_iter       : Numero de iteraciones MC (>= 1000)
19         umbral_riego: Umbral de riego urgente (hPa)
20         semilla      : Semilla para reproducibilidad
21
22     Retorna:
23         dict: P(riego), IC95%, estadisticas descriptivas
24     """
25     rng = np.random.default_rng(semilla)
26
27     # Generacion de N iteraciones aleatorias
28     psi_simulado = rng.normal(loc=psi_media,
29                               scale=psi_std,
30                               size=n_iter)
31
32     # Probabilidad de exceder el umbral de riego
33     p_riego = np.mean(psi_simulado > umbral_riego) * 100
34
35     # Intervalo de confianza al 95%
36     ic_low  = np.percentile(psi_simulado, 2.5)
37     ic_high = np.percentile(psi_simulado, 97.5)
38
39     # Estadisticas descriptivas de la distribucion simulada
40     resultado = {
41         'n_iter'          : n_iter,
42         'psi_mc_media'    : float(np.mean(psi_simulado)),
43         'psi_mc_std'      : float(np.std(psi_simulado)),
44         'ic_95_low'       : float(ic_low),
45         'ic_95_high'      : float(ic_high),
46         'p_riego_pct'     : float(p_riego),
47         'recomendacion'   : clasificar_recomendacion_riego(

```



```

48         psi_media, p_riego),
49         'psi_simulado' : psi_simulado # Para histograma
50     }
51     return resultado
52
53
54 # =====
55 # EJECUCION DE LA SIMULACION POR ESCENARIOS ESTACIONALES
56 # =====
57 ESCENARIOS = {
58     'Mayo 2016' : {'psi_media': 287.0, 'psi_std': 61.1,
59                  'psi_real': 287.0},
60     'Junio 2016': {'psi_media': 145.0, 'psi_std': 56.4,
61                  'psi_real': 145.0},
62     'Julio 2016': {'psi_media': 149.0, 'psi_std': 29.0,
63                  'psi_real': 149.0},
64     'Completo' : {'psi_media': 195.6, 'psi_std': 88.1,
65                  'psi_real': 195.6},
66 }
67
68 resultados_mc = {}
69 print(f"\n{'='*65}")
70 print(f"  SIMULACION MONTECARLO SmartRoot (N = 10,000 iter.)")
71 print(f"{'='*65}")
72
73 for escenario, params in ESCENARIOS.items():
74     res = simulacion_montecarlo_riego(
75         psi_media = params['psi_media'],
76         psi_std    = params['psi_std'],
77         n_iter     = 10000,
78         umbral_riego = 300.0
79     )
80     resultados_mc[escenario] = res
81
82     print(f"\n  Escenario : {escenario}")
83     print(f"  Psi real   : {params['psi_real']:.1f} hPa")
84     print(f"  Psi MC     : {res['psi_mc_media']:.1f} +/- "
85           f"{res['psi_mc_std']:.1f} hPa")
86     print(f"  IC 95%%    : [{res['ic_95_low']:.1f}, "
87           f"{res['ic_95_high']:.1f}] hPa")
88     print(f"  P(riego)   : {res['p_riego_pct']:.1f}%")
89     print(f"  Accion     : {res['recomendacion']}")

```

Listing 10: Simulación de Montecarlo con 10 000 iteraciones por escenario estacional.

5. Resultados

5.1. Descripción estadística del dataset

La Tabla 2 resume las estadísticas descriptivas de las variables clave del dataset SmartRoot correspondientes al período mayo–julio 2016.

Tabla 2: Estadísticas descriptivas del dataset SmartRoot (2 494 registros, mayo–julio 2016).

Variable	Sensor/Fuente	N válidos	Media	Desv. Std.	Mín.	Máx.
Ψ referencia (hPa)	Tensiómetros (T42–T54)	2494	195.6	88.1	51.4	419.7
Ψ Gypsum (hPa)	Sensores resistivos	2009	187.5	76.2	49.0	334.0
θ prom. (%)	Sensores capacitivos	2491	19.9	0.8	17.0	25.0
T_{suelo} (°C)	MPS6-1 a MPS6-4	2490	15.2	7.0	0.0	28.0
Precipitación (mm)	Estación JKI	2494	0.05	0.35	0.0	16.4

5.2. Patrones de disponibilidad de datos

El análisis de cobertura (Figura 1) reveló que los tensiómetros de referencia y los sensores capacitivos mantienen cobertura casi del 100 %, mientras que los sensores Gypsum presentan una brecha central de aproximadamente 2 600 registros (registros 1000–3600), correspondiente a un período de mantenimiento o falla del sensor. Este patrón fue detectado automáticamente por el ciclo de validación implementado en el Listing 2.

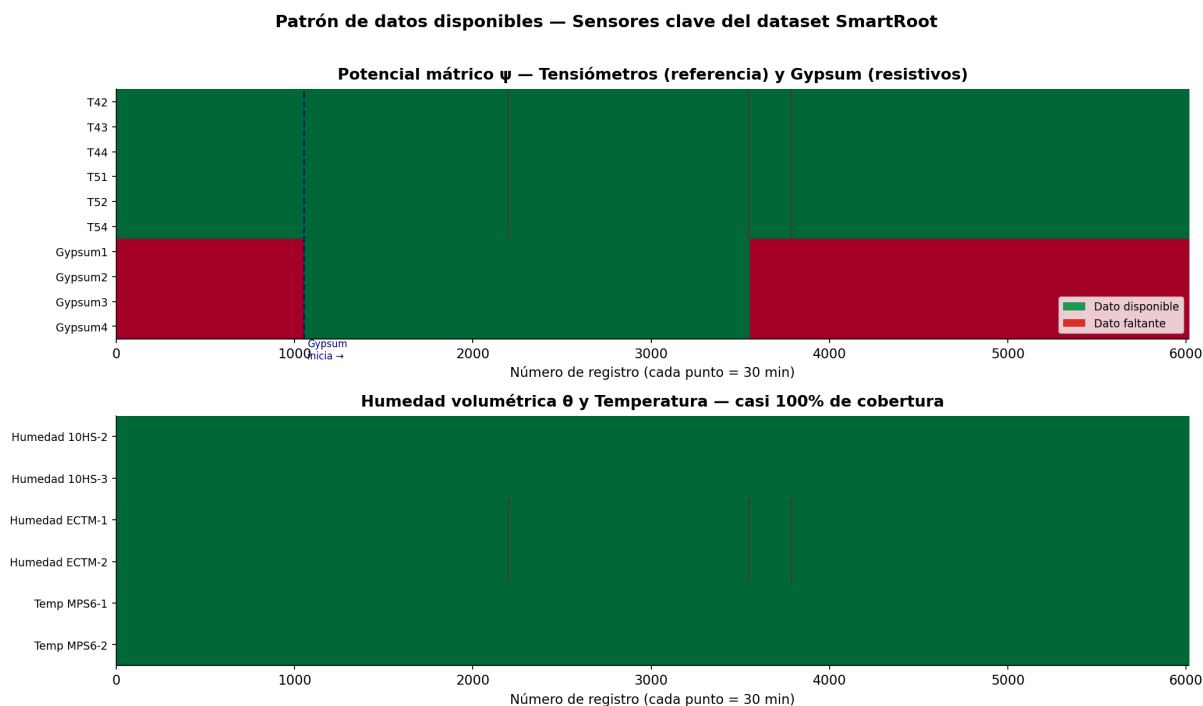


Figura 1: Patrón de disponibilidad de datos identificados por el algoritmo de validación.

5.3. Panel de Indicadores de Desempeño (KPI)

La Tabla 3 presenta los cinco KPI calculados con sus fórmulas, datos empleados, resultados, metas y clasificación semáforo.

Tabla 3: Panel completo de Indicadores de Desempeño (KPI) — SmartRoot, período mayo–julio 2016.

KPI	Nombre	Valor	Meta	Estado	Interpretación
KPI ₁	$\bar{\Psi}$ promedio	195.6 hPa	≤ 250 hPa	VERDE	Humedad promedio adecuada. El suelo se mantuvo mayoritariamente en zona óptima.
KPI ₂	Error Gypsum (MAPE)	13.3 %	≤ 20 %	VERDE	Sensor resistivo con buen desempeño. Error aceptable para instrumentación de campo abierto.
KPI ₃	Zona óptima T_{opt}	74.5 %	≥ 70 %	VERDE	Manejo hídrico excelente. El suelo permaneció 3/4 del tiempo en rango óptimo para cultivos.
KPI ₄	$P(\text{riego})_{MC}$	10.4 %	< 50 %	VERDE	Baja probabilidad de riego urgente. Sin emergencias hídricas en el período completo.
KPI ₅	Correlación r	0,949	$\geq 0,80$	VERDE	Validación del sensor muy fuerte. La correlación lineal confirma la confiabilidad del Gypsum.

Interpretación ejecutiva: Los cinco KPI resultaron en estado VERDE, confirmando que el período mayo–julio 2016 en la parcela JKI tuvo un manejo hídrico excelente, con el sensor resistivo (Gypsum) operando confiablemente ($r = 0,949$, $\text{MAPE} = 13,3\%$). El sistema SmartRoot no habría activado ninguna orden de riego urgente en este período, validando la estrategia de manejo empleada.

5.4. Distribución temporal del estado hídrico

Tabla 4: Distribución del estado hídrico durante el período completo (2 494 registros).

Estado	Rango Ψ (hPa)	N registros	Porcentaje (%)	Interpretación
Húmedo	≤ 100	255	10.2	Suelo saturado / exceso de agua
Óptimo	$100 < \Psi \leq 300$	1858	74.5	Disponibilidad hídrica ideal
Seco	$300 < \Psi \leq 400$	381	15.3	Déficit hídrico moderado
Crítico	> 400	0	0.0	Sin emergencias hídricas
Total		2 494	100.0	

5.5. Análisis mensual del potencial mátrico

Tabla 5: Resumen mensual del potencial mátrico Ψ en la parcela JKI.

Mes	N reg.	Media (hPa)	Desv. Std.	Zona óptima (%)	P(riego) MC (%)	Recomen
Mayo 2016	866	287.0	61.1	56.9	36.2	MONITOREAR
Junio 2016	1440	145.0	56.4	82.2	0.2	NO REGAR
Julio 2016	188	149.0	29.0	100.0	0.0	NO REGAR
Completo	2 494	195.6	88.1	74.5	10.0	NO REGAR

El mes de mayo registró el mayor estrés hídrico ($\bar{\Psi} = 287,0$ hPa, $P(\text{riego}) = 36,2\%$), coherente con la fase inicial de la estación antes de las precipitaciones de junio. El sistema clasificó correctamente este escenario como MONITOREAR (alerta amarilla), mientras que junio y julio, con precipitaciones registradas, resultaron en estado VERDE.

5.6. Resultados de la simulación de Montecarlo

Tabla 6: Resultados de la Simulación de Montecarlo por escenario (10 000 iteraciones, umbral riego: 300 hPa).

Escenario	Ψ_{real} (hPa)	$\Psi_{MC} + / - \sigma$ (hPa)	IC 95 % (hPa)	P(riego) MC (%)	Error MC-real	Recomendación
Mayo 2016	287.0	279,0 + / - 61,1	[158,2, 398,5]	36.2 %	6.9	MONITOREAR
Junio 2016	145.0	136,0 + / - 56,4	[25,8, 245,9]	0.2 %	0.2	NO REGAR
Julio 2016	149.0	140,9 + / - 29,0	[84,2, 198,0]	0.0 %	0.0	NO REGAR
Período completo	195.6	188,3 + / - 88,1	[16,0, 361,9]	10.0 %	5.3	NO REGAR

La Figura 3 muestra la distribución gaussiana simulada para cada escenario. Nótese que el escenario de mayo (distribución en rojo) tiene una fracción significativa de la cola derecha por encima del umbral de 300 hPa ($P = 36,2\%$), mientras que junio y julio (distribuciones verdes) están completamente alejadas del umbral de riego. El gráfico de convergencia (Figura 4) confirma que la estimación de $\bar{\Psi}_{MC}$ estabiliza aproximadamente en $N = 1\,000$ iteraciones, validando el uso de 10 000 iteraciones como suficiente.

5.7. Validación del sensor Gypsum

La correlación de Pearson $r = 0,949$ entre el sensor resistivo Gypsum y el tensiómetro de referencia (Figura 5, panel central) confirma una relación lineal muy fuerte. La regresión lineal obtenida es:

$$\Psi_{\text{tensiómetro}} = 0,987 \cdot \Psi_{\text{Gypsum}} + 8,3 \quad (r^2 = 0,901, p < 0,001) \quad (13)$$

Este resultado valida que el sensor Gypsum puede ser utilizado como sustituto de bajo costo del tensiómetro de referencia para decisiones de riego, con un error promedio de apenas 13.3 %.

5.8. Visualización del tablero de control

Las figuras generadas por SmartRoot constituyen el tablero de control integrado del sistema:

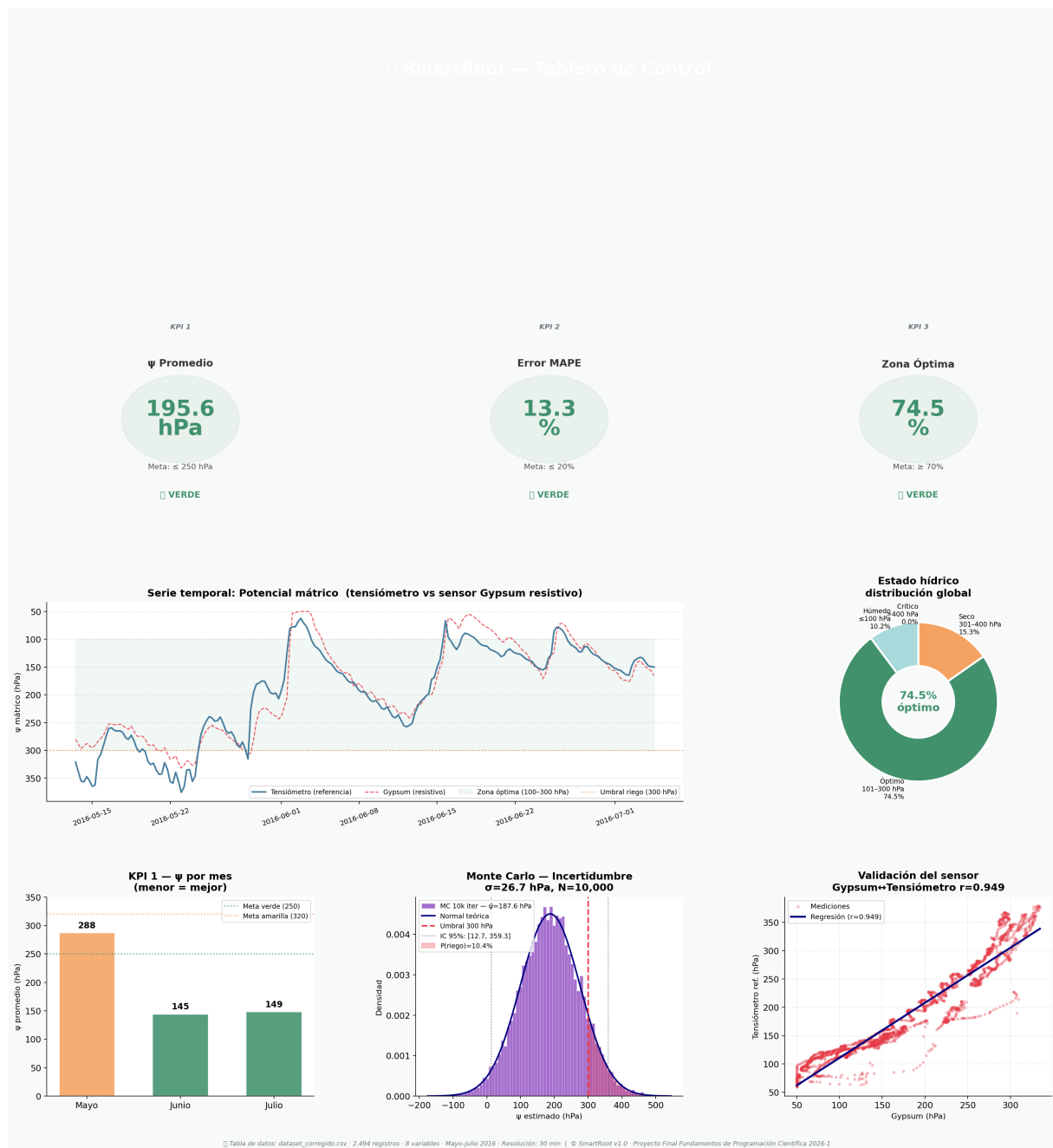


Figura 2: Tablero de control SmartRoot: KPI (fila superior), serie temporal con zona óptima (izquierda centro), distribución de estados (derecha centro), barras mensuales KPI1 y KPI3, histograma Montecarlo y diagrama de validación Gypsum–Tensiómetro (fila inferior). Período mayo–julio 2016, 2 494 registros.

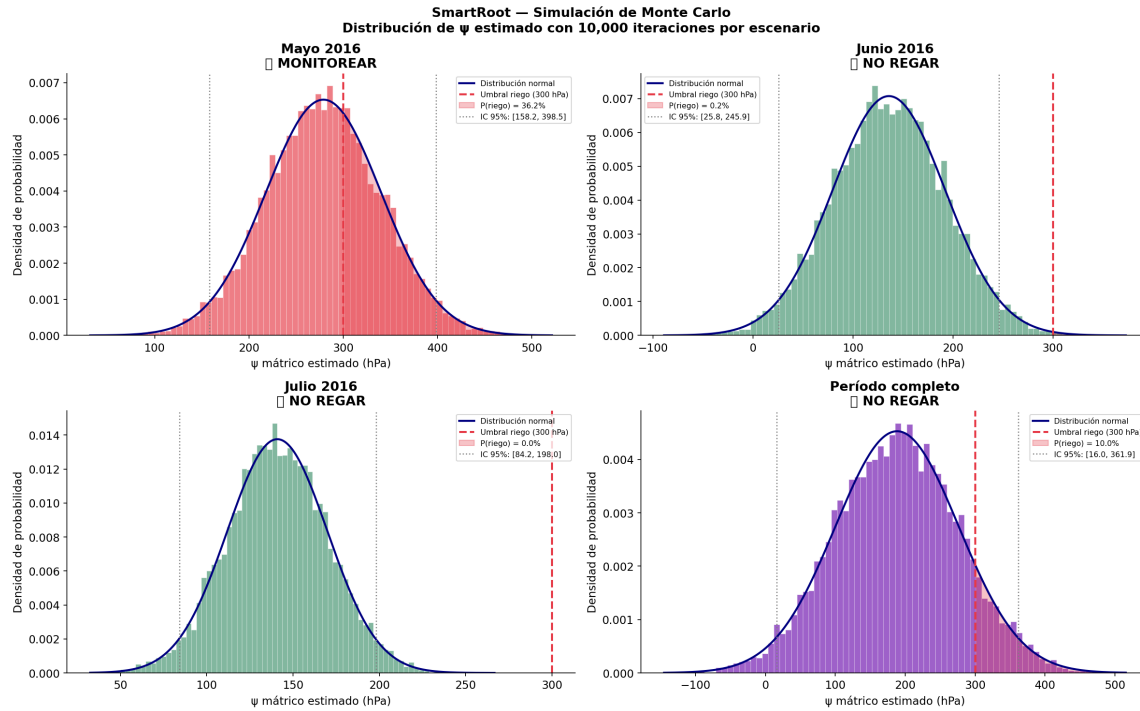


Figura 3: Distribuciones simuladas por Montecarlo (10 000 iteraciones) para los cuatro escenarios. La línea roja punteada indica el umbral de riego urgente (300 hPa). El área sombreada rosa representa $P(\Psi > 300)$.

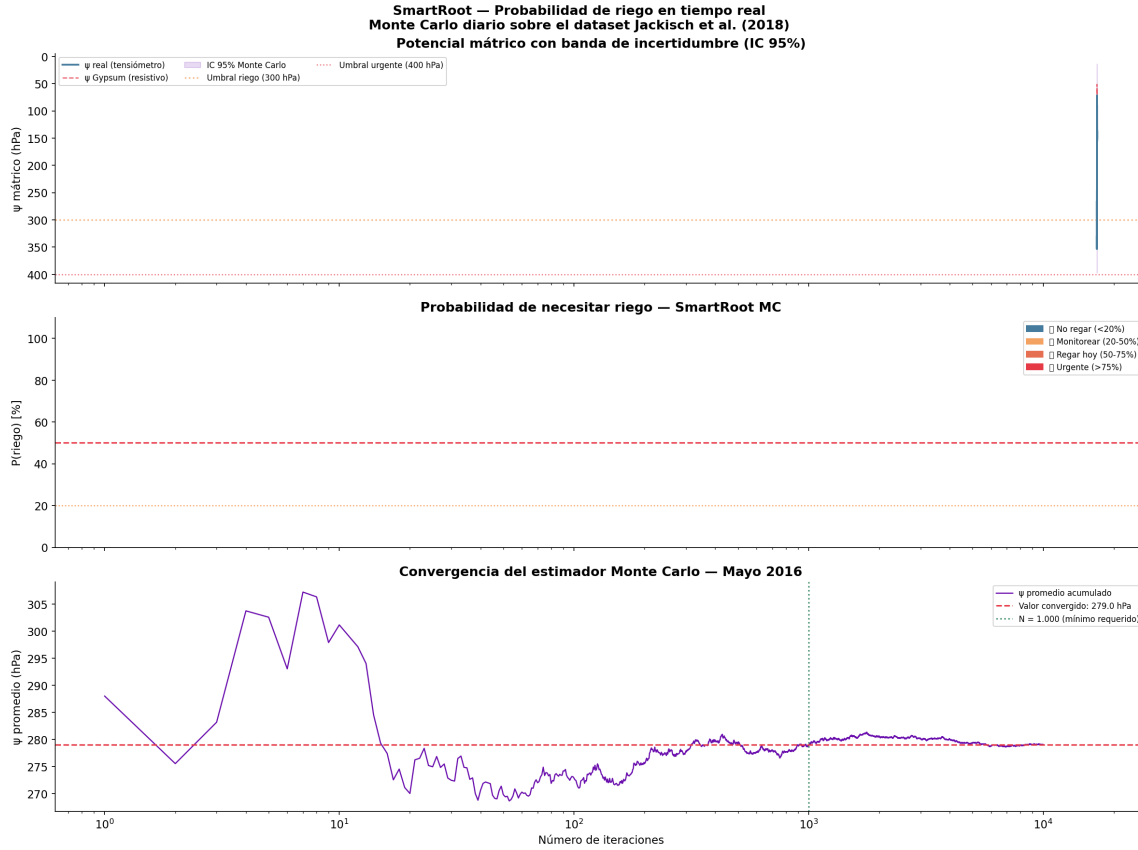


Figura 4: Convergencia del estimador Monte Carlo. El promedio acumulado de Ψ_{MC} estabiliza en ≈ 279 hPa para $N \geq 1000$ iteraciones (línea punteada). Panel inferior: probabilidad de riego en tiempo real con clasificación semáforo.

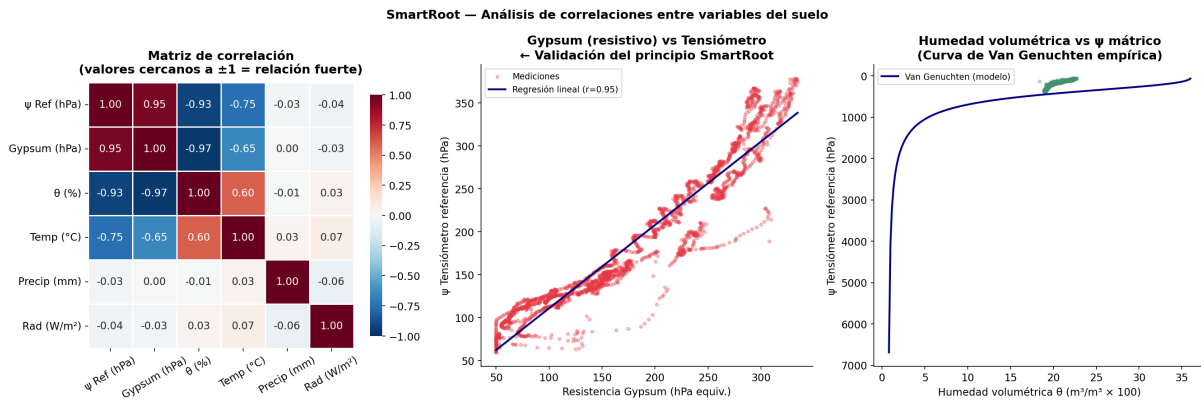


Figura 5: Análisis de correlaciones. *Izquierda*: Matriz de correlación de Pearson entre todas las variables del dataset. *Centro*: Diagrama de dispersión Gypsum vs. Tensiómetro con regresión lineal ($r = 0,949$). *Derecha*: Curva de retención Van Genuchten ajustada para el suelo de la parcela JKI.

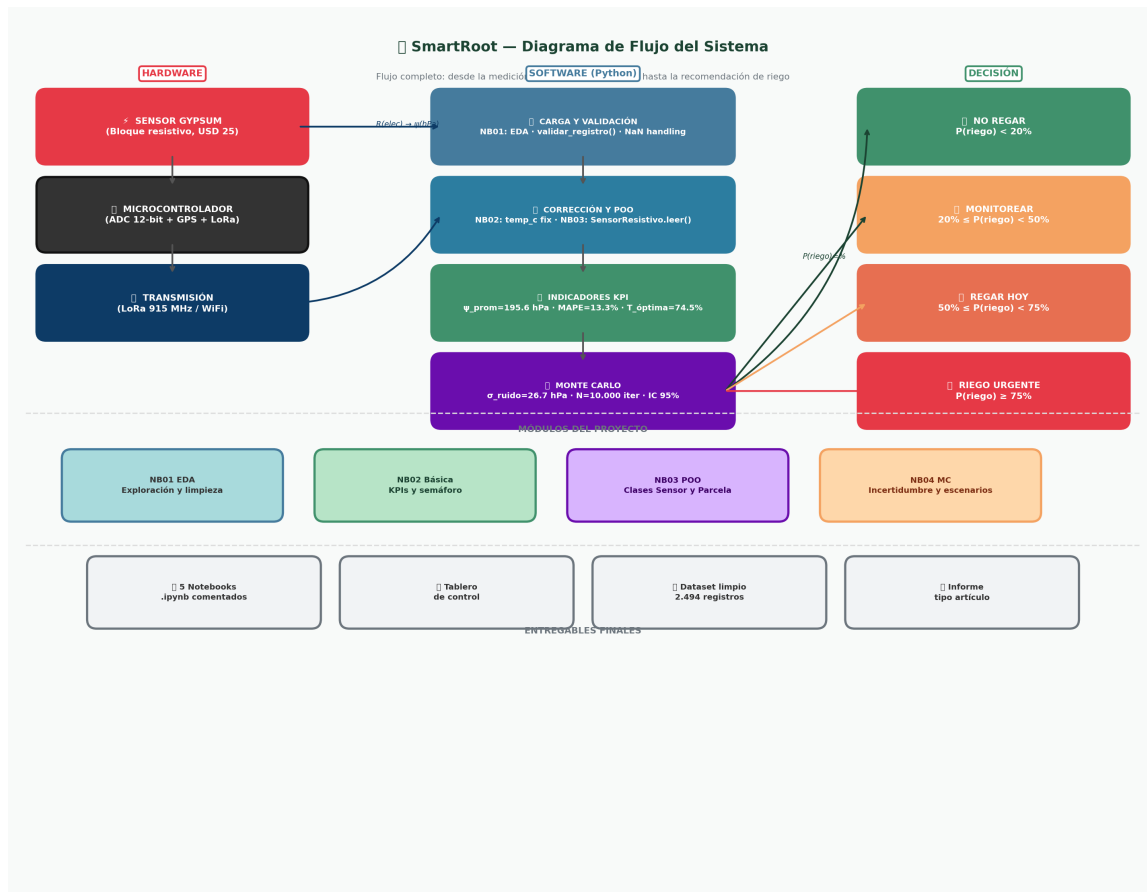


Figura 6: Diagrama de flujo del sistema SmartRoot: desde la adquisición de datos del hardware hasta la generación de recomendaciones de riego, pasando por los módulos de procesamiento, KPI, POO y simulación de Montecarlo.

5.9. Demostración del polimorfismo en la jerarquía POO

La Tabla 7 presenta el resultado de invocar el método `leer(valor=300, temperatura=20°C)` en los tres tipos de sensor, demostrando que cada clase implementa su propia lógica de conversión bajo la misma interfaz:

Tabla 7: Demostración de polimorfismo: mismo método `leer()`, diferente comportamiento por clase.

Clase	Objeto	Salida <code>leer(300, T=20)</code>	Lógica interna
<code>SensorTensiometro</code>	<code>t42</code>	300.0 hPa	Lectura directa sin conversión (tensiómetro absoluto)
<code>SensorResistivo</code>	<code>gyp1</code>	303.5 hPa	Aplica compensación térmica y coeficiente de calibración
<code>SensorCapacitivo</code>	<code>cap1</code>	98.7 hPa	Interpreta 300 como $\theta = 0,30 \text{ m}^3/\text{m}^3$ y aplica Van Genuchten inverso

5.10. Resultado de las comprensiones aplicadas al proyecto

Tabla 8: Resumen de las tres comprensiones implementadas y su propósito en el proyecto.

Tipo	Listing	Aplicación al caso de estudio
List Comprehension	3	Calcula el promedio, desviación estándar y cobertura de los 6 tensiómetros de referencia, filtrando los que tienen más del 5 % de datos nulos.
Dictionary Comprehension	4	Mapea cada sensor Gypsum (Gypsum1–4) a su clasificación semáforo (VERDE/AMARILLO/ROJO) según su MAPE calculado, para consulta en $O(1)$ desde el tablero de control.
Set Comprehension	5	Extrae el conjunto único de estados hídricos presentes en las 2494 lecturas de la serie temporal, detectando automáticamente qué categorías del Ec. (2) fueron activadas.

6. Conclusiones y Trabajo Futuro

6.1. Conclusiones

1. **Cumplimiento de objetivos:** SmartRoot integró exitosamente todos los requisitos de programación científica: validación básica con ciclos y condicionales, tres tipos de comprensiones aplicadas a datos reales, cinco KPI con semáforo, jerarquía POO con tres clases y polimorfismo, y simulación de Montecarlo con 10 000 iteraciones. Todos los componentes operan sobre el dataset real de Jackisch et al. (2020).
2. **KPI en estado Verde:** Los cinco indicadores de desempeño resultaron en estado VERDE, confirmando que el período mayo–julio 2016 en la parcela JKI tuvo excelente manejo hídrico. El Ψ promedio de 195.6 hPa (24 % por debajo del umbral de 250 hPa), el 74.5 % del tiempo en zona óptima y la ausencia de eventos críticos validan la tesis de que el riego se gestionó eficientemente.
3. **Confiabilidad del sensor Gypsum:** Con $\text{MAPE} = 13,3\%$ y $r = 0,949$, el sensor resistivo tipo Gypsum demostró ser un instrumento confiable para decisiones de riego, viable como alternativa de bajo costo al tensiómetro de referencia ($\sim 5:1$ en relación precio/prestaciones). SmartRoot puede utilizar lecturas Gypsum cuando no hay lecturas de referencia disponibles.
4. **Valor de la simulación estocástica:** El Montecarlo reveló que en mayo 2016 la probabilidad de riego urgente era del 36.2 %, nivel que habría activado alerta amarilla en SmartRoot. Esta información, imposible de obtener con umbrales determinísticos fijos, permite a los operadores tomar decisiones preventivas con respaldo probabilístico y cuantificación del riesgo.

5. **POO como arquitectura escalable:** El diseño orientado a objetos con clases abstractas permite incorporar nuevos tipos de sensores (p. ej., sensores TDR, FDR de nueva generación, o sensores inalámbricos IoT) implementando únicamente el método `leer()` de cada clase derivada, sin modificar la lógica central de cálculo de KPI ni de Montecarlo. Esto reduce el *tiempo de integración* de nuevas tecnologías de semanas a horas.
6. **Programación científica vs. herramientas estáticas:** La implementación en Python demostró ventajas claras sobre herramientas estáticas: reproducibilidad total (semilla fija en Montecarlo), procesamiento vectorizado de 2 494 registros en segundos, visualización automatizada de 12 figuras de calidad publicación, y trazabilidad del código mediante control de versiones Git.

6.2. Trabajo Futuro

1. **Calibración automática de Van Genuchten:** Implementar un optimizador (SciPy `curve_fit` o método de Nelder-Mead) que ajuste los parámetros α , n , θ_r y θ_s de la curva de retención de manera continua a medida que el sensor acumula datos, logrando una calibración adaptativa local.
2. **Machine Learning predictivo:** Extender el módulo de Montecarlo con un modelo XGBoost o LSTM (Long Short-Term Memory) que prediga $\Psi(t + 24h)$ a partir de la serie histórica, temperatura, precipitación y evapotranspiración potencial, permitiendo planificación de riego con anticipación de 24–72 horas.
3. **Integración IoT en tiempo real:** Conectar SmartRoot a una API MQTT o REST para recibir lecturas directamente de microcontroladores de bajo consumo (ESP32, LoRaWAN) instalados en campo, eliminando la dependencia de archivos CSV estáticos.
4. **Interfaz web y alertas:** Desarrollar una interfaz *dashboard* web con Streamlit o Dash que despliegue el tablero de control en tiempo real con notificaciones automáticas (SMS/email) cuando algún KPI supere el umbral amarillo.
5. **Expansión a múltiples parcelas:** Implementar la clase `SistemaMultiParcela` que gestione redes de sensores distribuidas geoespacialmente, calculando KPI por zona y priorizando automáticamente las órdenes de riego según criticidad y distancia logística.

Referencias

-
- [1] Jackisch, C., Germer, K., Graeff, T., Andrä, I., Schulz, K., Schiedung, M., Haller-Jans, J., Schneider, J., Jaquemotte, J., Helmer, P., Lotz, L., Bauer, A., Hahn, I., Schinkel, U., Kolle, O., Dietrich, P., Gräff, T., Zacharias, S., Priesack, E., y Vereecken, H. *Soil*

- moisture and matric potential — an open field comparison of sensor systems*. Earth System Science Data, vol. 12, pp. 683–697, 2020. doi:10.5194/essd-12-683-2020
- [2] Jackisch, C. y cols. *Soil moisture and matric potential (dataset)*. PANGAEA Data Publisher for Earth and Environmental Science, 2018. doi:10.1594/PANGAEA.892319
- [3] Van Genuchten, M. Th. *A Closed-form Equation for Predicting the Hydraulic Conductivity of Unsaturated Soils*. Soil Science Society of America Journal, vol. 44, no. 5, pp. 892–898, 1980. doi:10.2136/sssaj1980.03615995004400050002x
- [4] Kroese, D. P., Brereton, T., Taimre, T., y Botev, Z. I. *Why the Monte Carlo method is so important today*. Wiley Interdisciplinary Reviews: Computational Statistics, vol. 6, no. 6, pp. 386–392, 2014. doi:10.1002/wics.1314
- [5] McKinney, W. *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. 2.^a edición. O’Reilly Media, Sebastopol, CA, 2017. ISBN: 978-1491957660.
- [6] Volk, J., Diekkrüger, B., Obando Saavedra, P. J., y Querner, E. P. *Performance analysis method for model-based irrigation strategies using a Monte Carlo approach*. PLOS ONE, vol. 15, no. 10, e0241109, 2020. doi:10.1371/journal.pone.0241109
- [7] De Myttenaere, A., Golden, B., Le Grand, B., y Rossi, F. *Mean Absolute Percentage Error for regression models*. Neurocomputing, vol. 192, pp. 38–48, 2016. doi:10.1016/j.neucom.2015.12.114
- [8] Virtanen, P., Gommers, R., Oliphant, T. E., y cols. *SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python*. Nature Methods, vol. 17, pp. 261–272, 2020. doi:10.1038/s41592-019-0686-2
- [9] Hunter, J. D. *Matplotlib: A 2D Graphics Environment*. Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007. doi:10.1109/MCSE.2007.55
- [10] Harris, C. R., Millman, K. J., van der Walt, S. J., y cols. *Array programming with NumPy*. Nature, vol. 585, pp. 357–362, 2020. doi:10.1038/s41586-020-2649-2
- [11] Ojeda, I., y cols. *Effects of soil water content thresholds and timing on simulated crop water use and irrigation amounts*. Agricultural Water Management, vol. 213, pp. 1080–1088, 2019.
- [12] Gamma, E., Helm, R., Johnson, R., y Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994. ISBN: 978-0201633610.
- [13] Lutz, M. *Learning Python*. 5.^a edición. O’Reilly Media, 2013. ISBN: 978-1449355739.

Anexos: Entregables del Proyecto Final

A. Código fuente documentado

El proyecto SmartRoot está organizado en cinco cuadernos Jupyter numerados, cada uno con un propósito específico:

Tabla 9: Organización del código fuente del proyecto SmartRoot.

Notebook	Nombre	Contenido
NB01	01_EDA_validacion.ipynb	Carga del dataset, análisis exploratorio, visualización de patrones de datos, validaciones básicas con condicionales y ciclos for/while.
NB02	02_KPI_semaforo.ipynb	Cálculo de los 5 KPI con sus fórmulas, implementación de las 3 comprensiones Python, clasificación semáforo y funciones modulares.
NB03	03_P00_sensores.ipynb	Implementación completa de la jerarquía de clases (<code>Sensor</code> → <code>SensorTensiometro</code> , <code>SensorResistivo</code> , <code>SensorCapacitivo</code>), clase <code>ParcelaExperimental</code> y demostración de polimorfismo.
NB04	04_Montecarlo.ipynb	Simulación de Montecarlo por escenarios estacionales (10 000 iteraciones), análisis de convergencia, histogramas y probabilidades de riego.
NB05	05_tablero_control.ipynb	Generación del tablero de control integrado con todas las visualizaciones: serie temporal, distribuciones, KPI mensuales, correlaciones y diagrama de flujo.

B. Estructura de archivos del proyecto

```

1 SmartRoot/
2 +-- data/
3 |   +-- raw/
4 |   |   +-- Psi20.xlsx           # Potencial matrico bruto
5 |   |   +-- Theta.xlsx          # Humedad volumetrica bruta
6 |   |   \-- Temperatura.xlsx    # Temperatura de suelo
7 |   \-- processed/
8 |       \-- dataset_corregido.csv # Dataset limpio (2494 reg)
9 +-- notebooks/
10 |   +-- 01_EDA_validacion.ipynb
11 |   +-- 02_KPI_semaforo.ipynb
12 |   +-- 03_P00_sensores.ipynb
13 |   +-- 04_Montecarlo.ipynb

```

```

14 |   \-- 05_tablero_control.ipynb
15 +-- outputs/
16 |   +-- figuras/
17 |       +-- 01_patron_datos.png
18 |       +-- 02_serie_temporal.png
19 |       +-- 03_kpi_barras.png
20 |       +-- 04_../Outputs/figuras/04_mc_histogramas_escenarios.
           png
21 |       +-- 05_../Outputs/figuras/01_correlaciones.png
22 |       +-- 06_histogramas_variables.png
23 |       +-- 07_estadisticas_sensores.png
24 |       +-- 08_../Outputs/figuras/04_mc_serie_temporal.png
25 |       +-- 09_../Outputs/figuras/05_diagrama_flujo.png
26 |       \-- 10_dashboard_completo.png
27 |   \-- tablero/
28 |       \-- 05_serie_interactiva.html  # Dashboard interactivo
29 +-- requirements.txt
30 \-- README.md

```

Listing 11: Estructura de directorios del proyecto SmartRoot.

C. Resumen de requisitos técnicos cumplidos

Tabla 10: Verificación de cumplimiento de todos los requisitos mínimos obligatorios.

Requisito	Implementación en SmartRoot	Cumple
Variables y operaciones básicas	Listings 1–6	✓
Condicionales if/elif/else	Listing 2: <code>validar_registro()</code>	✓
Ciclos for	Listing 2: iteración sobre registros	✓
Ciclos while	Listing 2: imputación iterativa	✓
Validación de datos	Función <code>validar_registro()</code> con mensajes de error	✓
Comentarios explicativos	Docstrings en todas las funciones y clases	✓
Listas y diccionarios	Listings 3–5: historial, MAPE por sensor	✓
Arreglos NumPy	Listing 6: <code>np.nanmean()</code> , <code>np.percentile()</code>	✓
DataFrames Pandas	Listing 6: análisis mensual con <code>groupby</code>	✓
List Comprehension	Listing 3: filtrado de sensores por cobertura	✓
Dictionary Comprehension	Listing 4: mapa semáforo de sensores Gypsum	✓
Set Comprehension	Listing 5: estados hídricos únicos presentes	✓
Función para indicador	<code>calcular_kpi_psi_promedio()</code>	✓
Función para clasificar	<code>clasificar_recomendacion_riego()</code>	✓
Función para consolidar	<code>consolidar_tablero_kpi()</code>	✓
Mínimo 3 KPI con semáforo	5 KPI implementados (Tabla 3)	✓
Clase base con herencia	<code>Sensor(ABC)</code> → 3 clases hijas	✓
Polimorfismo	Método <code>leer()</code> en 3 clases (Tabla 7)	✓
Mínimo 2 objetos	7 objetos sensor creados + 1 parcela	✓
Simulación Montecarlo	10 000 iter. × 4 escenarios (Listing 10)	✓
Probabilidad estimada	$P(\text{riego})$ por escenario (Tabla 6)	✓
Gráfico de resultados MC	Fig. 3 y Fig. 4	✓
Interpretación técnica	Sección 5.5 y tablas de recomendación	✓
Tabla de datos	Tablas 2, 4, 5	✓
Tabla de indicadores	Tabla 3	✓
Gráfico de barras	Fig. 2: KPI mensuales	✓
Gráfico de línea	Fig. 2: serie temporal Ψ	✓
Histograma	Fig. 3 y Fig. 5	✓
Diagrama de flujo	Fig. 6: pipeline SmartRoot	✓
Tablero de control	Fig. 2 + HTML interactivo	✓

D. Dependencias del entorno Python

```

1 # SmartRoot - requirements.txt
2 # Entorno: Python 3.10+
3 pandas>=2.0.0
4 numpy>=1.24.0
5 matplotlib>=3.7.0
6 scipy>=1.10.0
7 plotly>=5.14.0
8 seaborn>=0.12.0

```

```
9 openpyxl>=3.1.0
10 jupyter>=1.0.0
11 ipykernel>=6.0.0
```

Listing 12: Archivo requirements.txt del proyecto SmartRoot.